PRODUCTION ENGINEERING BLUEPRINT

Hyper-Personalized Banking Ads & Next-Best-Action (NBA) System

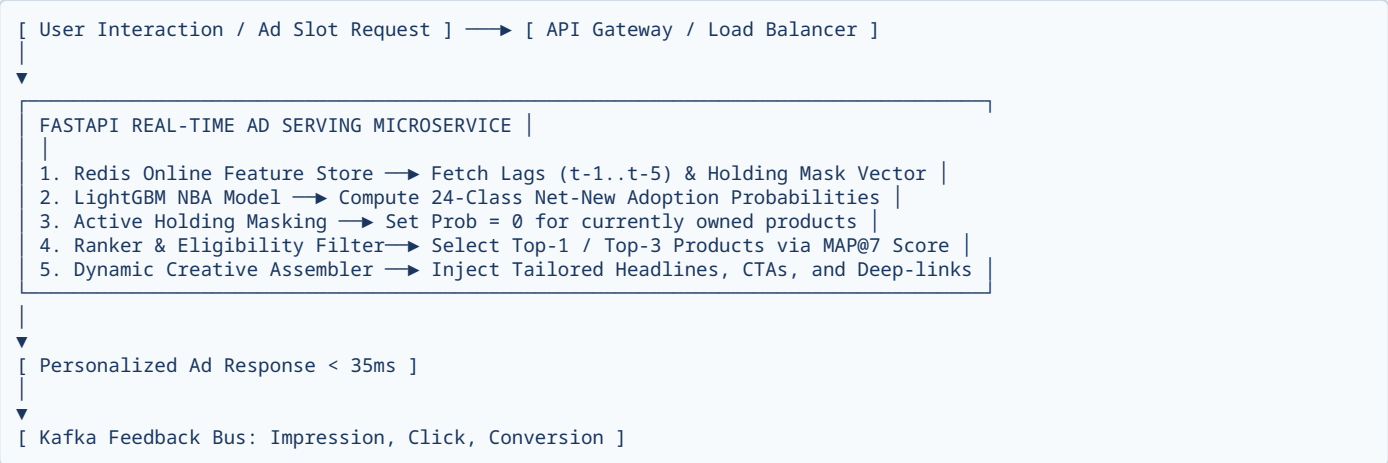
Industrial End-to-End Implementation: Santander Time-Series Modeling, MAP@7 Optimization, Dynamic LLM Copy, and Sub-35ms FastAPI/Redis Serving

Dataset: Santander Product Recommendation | **Core Algorithm:** LightGBM Multi-Class + Guardrails | **Evaluation:** MAP@7 & Ad Conversion Lift | **Latency SLA:** < 35ms p99

1. System Overview & Dual-Engine Architecture

Modern hyper-personalized financial advertising requires two tightly coupled systems operating in real-time:

1. **Predictive Next-Best-Action (NBA) Engine:** Scores the propensity of a user adopting any of 24 retail banking products in the upcoming month based on historical sequential holding trajectories.
2. **Dynamic Creative Optimization (DCO) & Assembly Layer:** Selects the optimal headline, visual banner, dynamic value hook, and deep-link call-to-action tailored to the user demographic and propensity profile.



2. Santander Dataset Schema & Problem Formulation

The Santander dataset contains 17 monthly snapshots across 1.2M customers and 24 financial products.

> **Formal Target Definition (Net-New Acquisition Only):**
> Predicting static ownership produces severe bias towards legacy accounts. The target for customer i, product k at month t is strictly:
>
> $$y_{i,t,k} = max(0, P_{i,t,k} - P_{i,t-1,k})$$
>
> If $\Sigma y_{i,t,k} = 0$, no product was added, and the record is excluded from positive multiclass training.

The 24 Target Banking Products

Product Code	Commercial Name	Product Code	Commercial Name
`ind_ahor_fin_ult1`	Saving Account	`ind_ecue_fin_ult1`	e-Account (Digital)
`ind_aval_fin_ult1`	Guarantees	`ind_fond_fin_ult1`	Mutual Funds
`ind_cco_fin_ult1`	Current Accounts	`ind_hip_fin_ult1`	Mortgage Loan
`ind_cder_fin_ult1`	Derivada Accounts	`ind_plan_fin_ult1`	Pensions Plan
`ind_cno_fin_ult1`	Payroll Account	`ind_pres_fin_ult1`	Consumer Loans
`ind_ctju_fin_ult1`	Junior Account	`ind_reca_fin_ult1`	Taxes Direct Debit

Product Code	Commercial Name	Product Code	Commercial Name
`ind_ctma_fin_ult1`	Mas Particular	`ind_tjcr_ult1`	Credit Cards
`ind_ctop_fin_ult1`	Particular Account	`ind_valo_fin_ult1`	Securities / Stocks
`ind_ctpp_fin_ult1`	Particular Plus	`ind_viv_fin_ult1`	Home Account
`ind_deco_fin_ult1`	Short-term Deposits	`ind_nomina_ult1`	Payroll Inflow
`ind_deme_fin_ult1`	Medium-term Deposits	`ind_nom_pens_ult1`	Pension Inflow
`ind_dela_fin_ult1`	Long-term Deposits	`ind_recibo_ult1`	Direct Debit (Bills)

3. Feature Engineering Architecture

Predictive strength relies on temporal lags and customer velocity:

- **Multi-Month Lags:** Ownership states across all 24 products at $t-1, t-2, t-3, t-4, t-5$ (120 features).
- **Transition Velocity:** Number of products added or dropped in the preceding 3 and 6 months.
- **Demographic Signals:** Gross household income (`renta`) imputed by provincial medians, age, customer seniority (`antiguedad`), and customer segment (01-VIP, 02-Retail, 03-University).
- **Seasonality:** Calendar month indicator capturing bonus disbursements (June/December) and tax cycles.

4. Python Training & MAP@7 Evaluation Pipeline

```
import polars as pl
import numpy as np
import lightgbm as lgb
from typing import List

TARGET_PRODUCTS = [
    "ind_ahor_fin_ult1", "ind_aval_fin_ult1", "ind_cco_fin_ult1", "ind_cder_fin_ult1",
    "ind_cno_fin_ult1", "ind_ctju_fin_ult1", "ind_ctma_fin_ult1", "ind_ctop_fin_ult1",
    "ind_ctpp_fin_ult1", "ind_deco_fin_ult1", "ind_deme_fin_ult1", "ind_dela_fin_ult1",
    "ind_ecue_fin_ult1", "ind_fond_fin_ult1", "ind_hip_fin_ult1", "ind_plan_fin_ult1",
    "ind_pres_fin_ult1", "ind_reca_fin_ult1", "ind_tjcr_ult1", "ind_valo_fin_ult1",
    "ind_viv_fin_ult1", "ind_nomina_ult1", "ind_nom_pens_ult1", "ind_recibo_ult1"
]

def compute_map_at_7(actuals: List[List[int]], preds: np.ndarray) -> float:
    scores = []
    for act, pred in zip(actuals, preds):
        if not act:
            continue
        top7 = np.argsort(pred)[::-1][:7]
        score, hits = 0.0, 0.0
        for i, p in enumerate(top7):
            if p in act and p not in top7[:i]:
                hits += 1.0
                score += hits / (i + 1.0)
        scores.append(score / min(len(act), 7))
    return float(np.mean(scores)) if scores else 0.0

def train_production_nba_model(X_train: np.ndarray, y_train: np.ndarray,
                               X_val: np.ndarray, y_val: np.ndarray):
    train_ds = lgb.Dataset(X_train, label=y_train)
    val_ds = lgb.Dataset(X_val, label=y_val, reference=train_ds)
    params = {
        "objective": "multiclass",
        "num_class": 24,
        "metric": "multi_logloss",
        "boosting_type": "gbdt",
        "learning_rate": 0.04,
        "num_leaves": 96,
        "feature_fraction": 0.8,
        "bagging_fraction": 0.8,
        "bagging_freq": 1,
        "verbosity": -1,
        "n_jobs": -1
    }
    booster = lgb.train(
        params, train_ds, num_boost_round=800,
        valid_sets=[train_ds, val_ds],
        callbacks=[lgb.early_stopping(50)]
    )
    booster.save_model("nba_lightgbm_production.txt")
    return booster
```

5. Dynamic Ad Creative & Dynamic LLM Copy Engine

When the NBA model predicts a top target product, the creative engine maps user segment attributes to targeted copy and call-to-action hooks:

Target Product	Customer Segment	Dynamic Personalized Headline	CTA Button
`ind_tjcr_ult1` (Credit Card)	02 - PARTICULARES	"Earn 5% cashback on dining and groceries with zero annual fees."	Apply in 1-Click
`ind_hip_fin_ult1` (Mortgage)	01 - TOP VIP	"Exclusive fixed mortgage rates from 2.45% APR for your new estate."	Calculate Rate

Target Product	Customer Segment	Dynamic Personalized Headline	CTA Button
`ind_cno_fin_ult1` (Payroll)	03 - UNIVERSITARIO	"Direct deposit your paycheck & receive an instant €150 bonus."	Switch Account
`ind_plan_fin_ult1` (Pensions)	Senior / Mature	"Maximize your annual tax savings with Santander Managed Retirement."	Start Plan

> **Generative LLM Copy Prompt:** For rare personas, an asynchronous LLM worker synthesizes ad copy:
> `System Prompt: "You are a senior banking copywriter. Generate a concise 12-word ad headline and a 3-word CTA tailored for customer segment {segment}, age {age}, pitching product {product_name}."`

6. Real-Time Serving: FastAPI & Redis Microservice

```
import time, json, redis
import numpy as np
import lightgbm as lgb
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="Santander Personalized Ad Engine", version="1.0.0")
booster = lgb.Booster(model_file="nba_lightgbm_production.txt")
r = redis.Redis(host="localhost", port=6379, db=0, decode_responses=True)

PRODUCT_REGISTRY = {
    0: {"code": "ind_ahor_fin_ult1", "title": "Santander High Yield Savings", "cta": "Open Savings"},
    4: {"code": "ind_cno_fin_ult1", "title": "Payroll Account (€150 Bonus)", "cta": "Switch Salary"},
    12: {"code": "ind_ecue_fin_ult1", "title": "Zero-Fee Digital e-Account", "cta": "Open Account"},
    14: {"code": "ind_hip_fin_ult1", "title": "Fixed Rate Home Mortgage", "cta": "Check Eligibility"},
    15: {"code": "ind_plan_fin_ult1", "title": "Retirement Pension Growth", "cta": "Start Investing"},
    18: {"code": "ind_tjcr_ult1", "title": "Santander Gold 5% Cashback Card", "cta": "Claim Card"},
    23: {"code": "ind_recibo_ult1", "title": "Automated Utility Direct Debit", "cta": "Setup Bills"}
}

class AdRequest(BaseModel):
    customer_id: int
    channel: str = Field(default="app_home_banner")

class AdResponse(BaseModel):
    customer_id: int
    product_code: str
    headline: str
    cta: str
    deep_link: str
    score: float
    latency_ms: float

@app.post("/v1/ad/serve", response_model=AdResponse)
async def serve_ad(req: AdRequest):
    t_start = time.perf_counter()
    raw_data = r.hgetall(f"user:{req.customer_id}")
    if not raw_data:
        raise HTTPException(status_code=404, detail="Customer not found in Feature Store")
    features = np.array(json.loads(raw_data["features"]), dtype=np.float32).reshape(1, -1)
    holdings = np.array(json.loads(raw_data["holdings"]), dtype=np.int32)
    scores = booster.predict(features)[0]
    scores[holdings == 1] = 0.0  # Guardrail: Mask active holdings
    recent_ads = r.smembers(f"freq_cap:{req.customer_id}")
    for idx, meta in PRODUCT_REGISTRY.items():
        if meta["code"] in recent_ads:
            scores[idx] *= 0.15
    top_idx = int(np.argmax(scores))
    product_meta = PRODUCT_REGISTRY.get(top_idx, PRODUCT_REGISTRY[18])
    latency = (time.perf_counter() - t_start) * 1000.0
    return AdResponse(
        customer_id=req.customer_id,
        product_code=product_meta["code"],
        headline=f"Exclusive offer for you: {product_meta['title']}",
        cta=product_meta["cta"],
        deep_link=f"santander://ad/open?prod={product_meta['code']}&channel={req.channel}",
        score=round(float(scores[top_idx]), 4),
        latency_ms=round(latency, 2)
    )
```

7. Production Guardrails & MLOps Lifecycle

- **Hard Active Ownership Filtering:** Strictly mask out currently active accounts to avoid burning ad spend on products the user already holds.
- **Credit Eligibility Filters:** Apply rule-based overrides post-inference (e.g. minimum income criteria for premium cards, age restrictions for retirement pensions).

- **Fatigue Frequency Capping:** Cap impressions at a maximum of 3 exposures per product per customer within a rolling 7-day window.
- **Kafka Feedback & Online Monitoring:** Stream impression, click, and conversion events back to an event bus to track CTR lifts against an A/B test baseline and monitor Population Stability Index (PSI) drift.